

Answers for Week 11

1. Job Script Workflows

To begin with, we will warm by just making some simple job scripts. For all exercises in this section, your job scripts must submit to the 'hpc' queue, use 1 core on 1 node and specify a job name, output file and error file.

1. **Autolab** Make a job script that creates a job array with job indices running from 1 to 10.

Answer: Autolab exercise - answer omitted

2. **Autolab** Make a job script that creates a job array with job indices 2, 29, 71, 73 and 127.

Answer: Autolab exercise - answer omitted

3. **Autolab** Make a job script that waits until another job with id 1234567 exits successfully.

Answer:

4. Assume we have the following output from `bstat` for a job array which is already submitted:

```

1 $ bstat
2 JOBID      USER    QUEUE    JOB_NAME    NALLOC  STAT  START_TIME
3 21241475   patmjen hpc      array[1]      1  RUN  Apr  9 13:40
4 21241475   patmjen hpc      array[2]      1  RUN  Apr  9 13:40
5 21241475   patmjen hpc      array[3]      1  RUN  Apr  9 13:40
6 21241475   patmjen hpc      array[4]      1  RUN  Apr  9 13:40
7 21241475   patmjen hpc      array[5]      1  RUN  Apr  9 13:40

```

Make job script that creates a job array, where each job in the array waits for the corresponding job in the already submitted array to finish successfully.

Answer:

```
1 #!/bin/bash
2 #BSUB -J jobname[1-5]
3 #BSUB -q hpc
4 #BSUB -W 5
5 #BSUB -n 1
6 #BSUB -R "span[hosts=1]"
7 #BSUB -R "rusage[mem=512MB]"
8 #BSUB -w "done(array[*])"
9 #BSUB -o jobname_%J.out
10 #BSUB -e jobname_%J.err
```

5. **Autolab** Again, assume we have the following output from 'bstat' for a job array which is already submitted:

```
1 $ bstat
2 JOBID      USER      QUEUE      JOB_NAME      NALLOC  STAT  START_TIME
      ELAPSED
3 21241475   patmjen  hpc        array[1]        1  RUN   Apr  9 13:40
      0:00:09
4 21241475   patmjen  hpc        array[2]        1  RUN   Apr  9 13:40
      0:00:09
5 21241475   patmjen  hpc        array[3]        1  RUN   Apr  9 13:40
      0:00:09
6 21241475   patmjen  hpc        array[4]        1  RUN   Apr  9 13:40
      0:00:09
7 21241475   patmjen  hpc        array[5]        1  RUN   Apr  9 13:40
      0:00:09
```

Make a job script that waits until all elements of the job array have exited for any reason.

Answer: Autolab exercise - answer omitted.

2. Face Colors



In this exercise we will return to the [CelebA](#) dataset of celebrity images as our data source. As a way of summarizing the dataset, we will create a histogram over the colors in the images - more specifically, the [hue](#) values. The images are stored in subfolders at: `/dtu/projects/02613_2024/data/celeba/images`. Each subfolder contains around 1000 images. To speed up the computation, we will process each folder as a separate job (using a job array) and then aggregate the results into a combined histogram as a final job at the end.

Consider the following function, which extracts a histogram from an RGB stored as a (H, W, 3) NumPy array:

```
1 import numpy as np
2 from PIL import Image
3
4 def huehist(image):
5     bins = np.linspace(0, 255, 64 + 1)
6     hsv_image = np.asarray(Image.fromarray(image).convert('HSV'))
7     hue_values = hsv_image[:, :, 0].reshape(-1)
8     hue_hist = np.histogram(hue_values, bins)[0]
9     return hue_hist
```

1. Create a Python program which takes an index i as a command line argument. For the i th folder in `/dtu/projects/02613_2024/data/celeba/images`, the program must then:
 - a) Load every image in the folder. Hint: use [glob.glob](#) or [os.listdir](#) to get the files in a folder.
 - b) Compute the hue histogram for each image using the provided function.
 - c) Sum the histograms to get a combined histogram for the folder.
 - d) Save the summed histogram to a NumPy array named `subhist_<i>.npy`, where `<i>` is replaced with the value of the provided index i .

Answer:

```

1 import os
2 from os.path import join
3 import sys
4 import numpy as np
5 from PIL import Image
6
7 def huehist(image):
8     bins = np.linspace(0, 255, 64 + 1)
9     hsv_image = np.asarray(Image.fromarray(image).convert('HSV'))
10    hue_values = hsv_image[:, :, 0].reshape(-1)
11    hue_hist = np.histogram(hue_values, bins)[0]
12    return hue_hist
13
14 if __name__ == '__main__':
15     idx = int(sys.argv[1]) - 1 # Job arrays must be 1-indexed
16     base_path = '/dtu/projects/02613_2024/data/celeba/images'
17     folders = sorted(os.listdir(base_path))
18     images = sorted(os.listdir(join(base_path, folders[idx])))
19
20     hist = 0
21     for im in images:
22         image = Image.open(join(base_path, folders[idx], im))
23         image = np.array(image)
24         hist += huehist(image)
25
26     np.save(f"subhist_{idx}.npy", hist)

```

-
2. Create a Python program that loads the saved histogram arrays, sums them to a combined histogram for the entire dataset, plots the histogram using `plt.bar` and finally saves the histogram plot as an image.
-

Answer:

```

1 from glob import glob
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 if __name__ == '__main__':
6     histfiles = glob('subhist_*.npy')
7     hist = 0
8     for hf in histfiles:
9         hist += np.load(hf)
10    plt.bar(np.linspace(0, 255, 64), hist, width=4)
11    plt.savefig('histogram.png')

```

-
3. Make a job script that submits a job array where each job computes the hue histogram for the i th folder using the first Python program.
-

Answer:

```
1 #!/bin/bash
2 #BSUB -J subhist[1-203]
3 #BSUB -q hpc
4 #BSUB -W 15
5 #BSUB -n 1
6 #BSUB -R "span[hosts=1]"
7 #BSUB -R "rusage[mem=512MB]"
8 #BSUB -o batch_output/subhist_%I_%J.out
9 #BSUB -e batch_output/subhist_%I_%J.err
10
11 source /dtu/projects/02613_2024/conda/conda_init.sh
12 conda activate 02613
13
14 python hueidir.py $LSB_JOBINDEX
```

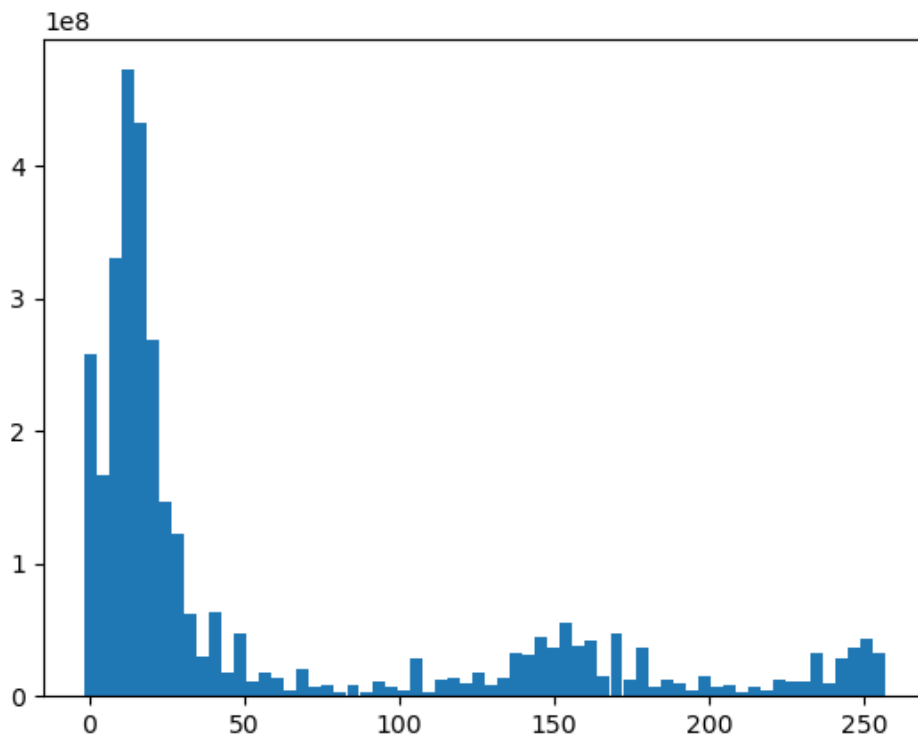
4. Make a job script that waits for all jobs in the previous job array to finish successfully and then runs the second Python program to compute and plot the final hue histogram.
-

Answer:

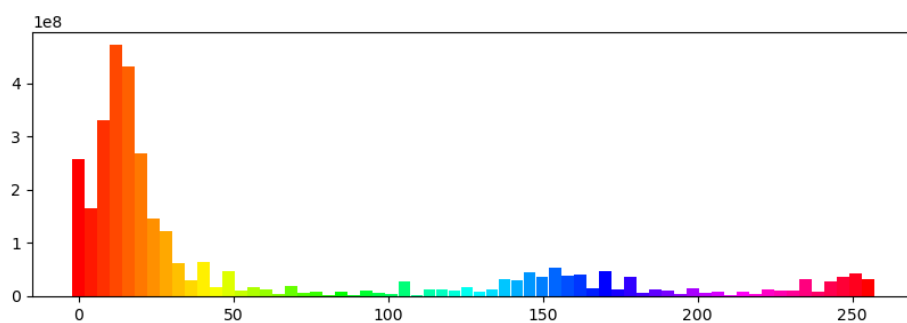
```
1 #!/bin/bash
2 #BSUB -J plothist
3 #BSUB -q hpc
4 #BSUB -W 5
5 #BSUB -n 1
6 #BSUB -w done(subhist)
7 #BSUB -R "span[hosts=1]"
8 #BSUB -R "rusage[mem=512MB]"
9 #BSUB -o batch_output/plothist_%J.out
10 #BSUB -e batch_output/plothist_%J.err
11
12 source /dtu/projects/02613_2024/conda/conda_init.sh
13 conda activate 02613
14
15 python plothuehist.py
```

5. Submit your jobs and compute the histogram.

Answer: Submitting the jobs, you should see the `subhist` jobs running and the `plothist` job remain pending until all the `subhist` jobs are done. This should give the following histogram:



Making the plotting code a bit more fancy, we can show the hue as part of the histogram:



The most used colors are the red hues, which makes sense since skin colors are from red and orange hues.

The plotting code for the fancy histogram is:

```
1 def plothuehist(h, bins=None, ax=None):
2     if bins is None:
3         bins = np.linspace(0, 255, 64)
4     if ax is None:
5         ax = plt.gca()
6     patches = ax.bar(bins, h, width=4).patches
7     h = bins / 255
8     s = np.ones_like(h)
9     v = np.ones_like(h)
10    hsv = (np.stack([h, s, v], axis=-1) * 255).astype(np.uint8)
11    rgb = np.asarray(Image.fromarray(hsv[None], mode='HSV').
12                      convert('RGB'))
12    rgb = rgb.squeeze() / 255.0
13    for i, patch in enumerate(patches):
14        patch.set_facecolor(rgb[i])
```
